Lab 8.1: Using the Data-Bound Dialog Box

In this lab, you will add, delete, and update records in a database table. You will also filter records in a recordset to display only records that meet criteria entered by the user.

To see a demonstration of the solution to this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 8.1 Demonstration.")

Estimated time to complete this lab: **40 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch08\Lab8.1\Ex01.

The solution code for this lab is located in the folder \\Labs\Ch08\Lab8.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Add, update, and delete records in a database table.
- Filter records in a recordset.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter.

Exercise 1: Adding, Deleting, and Updating Records

In this exercise, you will implement the **add**, **delete**, and **update** functions to manipulate records in a database table.

Exercise 2: Filtering Records

In this exercise, you will use the **Filter** property of the recordset to filter records that satisfy the specified criteria.

266 Exercise 1: Adding, Deleting, and Updating Records

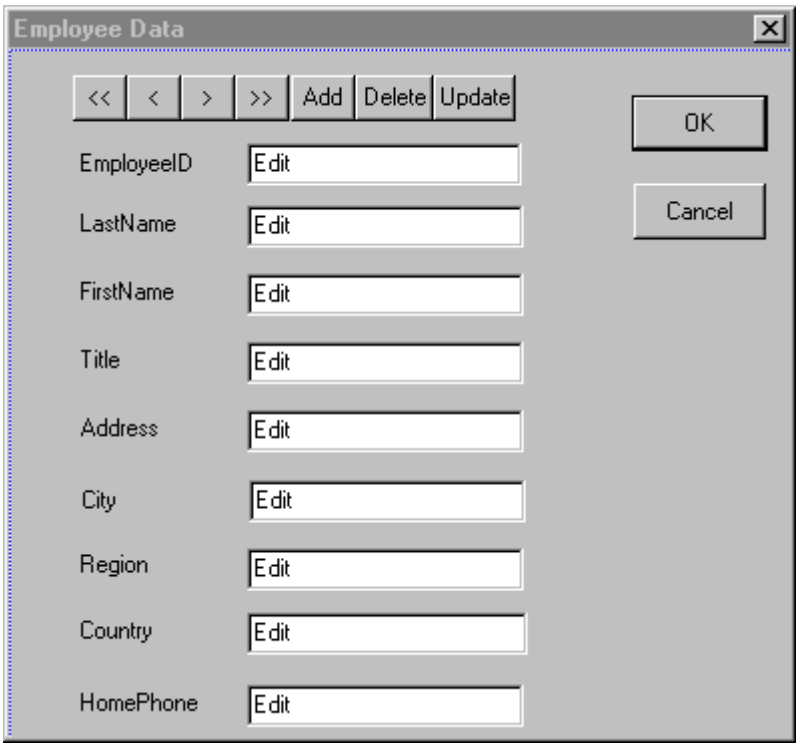
The code that forms the basis for this exercise is in \Labs\Ch08\Lab8.1\Ex01.

In this exercise, you will implement the **add**, **update**, and **delete** functions to manipulate records in a database table.

The base application has been generated by the Data Bound Dialog Box Wizard and already has handlers that enable the user to navigate through the records in a database table.

▶ Add controls to the CG_IDD_RECORDSET dialog box

- In the **CG_IDD_RECORDSET** dialog box, add three buttons to implement adding, deleting and updating records as shown in the following illustration.



Give the **Add**, **Delete**, and **Update** controls the IDs **ID_ADD**, **ID_DELETE**, and **ID_UPDATE** respectively.

Notes

▶ **Create message map entries for the Add, Delete, and Update buttons**

☐ 1. Open the CRsCgDlg.h file and add the following entries after the //{{AFX_MSG message:

```
afx_msg void OnAdd();
afx_msg void OnDelete();
afx_msg void OnUpdate();
```

☐ 2. Insert the following message map entries for ON_BN_CLICKED in the **CRsCgDlg** implementation file:

```
ON_BN_CLICKED(ID_ADD,OnAdd)
ON_BN_CLICKED(ID_DELETE,OnDelete)
ON_BN_CLICKED(ID_UPDATE,OnUpdate)
```

▶ **Implement the OnAdd button handler**

In this procedure, you will add records to a recordset by calling the **AddNew** function of the **_Recordset** class.

☐ 1. In RsCgDlg.cpp file, insert a handler for **OnAdd** as follows:

```
void CRsCgDlg::OnAdd()
{
```

☐ 2. Start a **try** block.

☐ 3. Invoke the **UpdateData** function with the argument **TRUE** to transfer the values from controls on the dialog box into the member variables corresponding to each column in the database table.

☐ 4. Declare two arrays of type **COleVariant** to store the names of columns in the table and the member variable values corresponding to each column in the table as follows:

```
COleVariant fld[9],val[9];
```

☐ 5. Store the names of columns in the table and member variable values in the arrays fld and val, respectively:

```
fld[0]="EmployeeID";
val[0]=m_lDlgEmployeeID;
fld[1]="LastName";
val[1]=m_strDlgLastName;
fld[2]="FirstName";
val[2]=m_strDlgFirstName;
fld[3]="Title";
val[3]=m_strDlgTitle;
fld[4]="Address";
val[4]=m_strDlgAddress;
fld[5]="City";
val[5]=m_strDlgCity;
fld[6]="Region";
val[6]=m_strDlgRegion;
fld[7]="Country";
val[7]=m_strDlgCountry;
fld[8]="HomePhone";
val[8]=m_strDlgHomePhone;
```

Notes

☐ 6. Declare and create two SafeArrays of variants as follows:

```
VARIANT rgf,rgv;
rgf.vt=VT_ARRAY|VT_VARIANT;
rgv.vt=VT_ARRAY|VT_VARIANT;
SAFEARRAYBOUND bound;
bound.cElements=9;
bound.lLbound=0;

rgv.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
if(rgv.parray==NULL)
    AfxThrowMemoryException();
rgf.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
if(rgf.parray==NULL)
    AfxThrowMemoryException();
```

☐ 7. Declare a variable of type **long** to store indexes for each dimension of the SafeArray as follows:

```
long rgIndices[1];
```

☐ 8. Assign the values stored in the arrays fld and val to SafeArray as follows:

```
for(int i=0; i<9;i++)
{
    rgIndices[0]=i;
    ::SafeArrayPutElement(rgf.parray, rgIndices, &fld[i]);
    ::SafeArrayPutElement(rgv.parray, rgIndices, &val[i]);
}
```

☐ 9. Invoke the **AddNew** function to add the record to the database table as follows:

```
m_pRs->AddNew(&rgf,&rgv);
```

☐ 10. Write the code for the **catch** handler as follows:

```
catch (_com_error &e)
{
    GenerateError(e.Error(), e.Description());
}
```

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CRsCgDlg::OnAdd()
{

    try
    {
        UpdateData(TRUE);

        COleVariant fld[9],val[9];
        fld[0]="EmployeeID";
        val[0]=m_lDlgEmployeeID;
        fld[1]="LastName";
        val[1]=m_strDlgLastName;
        fld[2]="FirstName";
        val[2]=m_strDlgFirstName;
        fld[3]="Title";
        val[3]=m_strDlgTitle;
        fld[4]="Address";
        val[4]=m_strDlgAddress;
        fld[5]="City";
        val[5]=m_strDlgCity;
        fld[6]="Region";
        val[6]=m_strDlgRegion;
        fld[7]="Country";
        val[7]=m_strDlgCountry;
        fld[8]="HomePhone";
        val[8]=m_strDlgHomePhone;

        VARIANT rgf,rgv;
        rgf.vt=VT_ARRAY|VT_VARIANT;
        rgv.vt=VT_ARRAY|VT_VARIANT;
        SAFEARRAYBOUND bound;
        bound.cElements=9;
        bound.lLbound=0;

        rgv.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgv.parray==NULL)
            AfxThrowMemoryException();
        rgf.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgf.parray==NULL)
            AfxThrowMemoryException();
```

(Code continues on following page.)

Notes

```
        long rgIndices[1];
        for(int i=0; i<9;i++)
        {
        rgIndices[0]=i;
        ::SafeArrayPutElement(rgf.parray, rgIndices, &fld[i]);
        ::SafeArrayPutElement(rgv.parray, rgIndices, &val[i]);
        }
        m_pRs->AddNew(&rgf,&rgv);

    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

▶ **Implement the OnDelete button handler**

In this procedure, you will use the **Delete** function to delete the current record of a recordset.

☐ 1. In RsCgDlg.cpp file, insert a handler for **OnDelete** as follows:

```
void CRsCgDlg::OnDelete()
{
```

☐ 2. Start a **try** block.

☐ 3. Invoke the **Delete** function to delete the record in the database table as follows:

```
m_pRs->Delete(adAffectCurrent);
```

☐ 4. Move to the next record because the record that you deleted is now invalid:

```
m_pRs->MoveNext();
if(m_pRs->EndOfFile)
    m_pRs->MoveLast();
RefreshBoundData();
```

☐ 5. Write the code for the **catch** handler as follows:

```
catch (_com_error &e)
{
    if(m_pRs->EndOfFile && m_pRs->BOF)
        BlankForm();
    else
        GenerateError(e.Error(), e.Description());
}
```

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CRsCgDlg::OnDelete()
{
    try
        {
            m_pRs->Delete(adAffectCurrent);
            m_pRs->MoveNext();
            if(m_pRs->EndOfFile)
                m_pRs->MoveLast();
            RefreshBoundData();
        }
        catch (_com_error &e)
        {
            if(m_pRs->EndOfFile && m_pRs->BOF)
                BlankForm();
            else
                GenerateError(e.Error(), e.Description());
        }

}
```

▶ **Implement the OnUpdate button handler**

In this procedure, you will use the **Update** function to edit the current record of a recordset. The code for the **OnUpdate** handler is similar to that for the **OnAdd** button handler. You will pass the names of the fields to be updated and the new values to the **Update** function.

- Invoke the **Update** function as follows:
  `m_pRs->Update(&rgf,&rgv);`

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CRsCgDlg::OnUpdate()
{

    try
    {
        UpdateData(TRUE);

        COleVariant fld[9],val[9];
        fld[0]="EmployeeID";
        val[0]=m_lDlgEmployeeID;
        fld[1]="LastName";
        val[1]=m_strDlgLastName;
        fld[2]="FirstName";
        val[2]=m_strDlgFirstName;
        fld[3]="Title";
        val[3]=m_strDlgTitle;
        fld[4]="Address";
        val[4]=m_strDlgAddress;
        fld[5]="City";
        val[5]=m_strDlgCity;
        fld[6]="Region";
        val[6]=m_strDlgRegion;
        fld[7]="Country";
        val[7]=m_strDlgCountry;
        fld[8]="HomePhone";
        val[8]=m_strDlgHomePhone;

        VARIANT rgf,rgv;
        rgf.vt=VT_ARRAY|VT_VARIANT;
        rgv.vt=VT_ARRAY|VT_VARIANT;
        SAFEARRAYBOUND bound;
        bound.cElements=9;
        bound.lLbound=0;

        rgv.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgv.parray==NULL)
            AfxThrowMemoryException();
        rgf.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgf.parray==NULL)
            AfxThrowMemoryException();
```

(Code continues on following page.)

Notes

```
        long rgIndices[1];
        for(int i=0; i<9;i++)
        {
        rgIndices[0]=i;
        ::SafeArrayPutElement(rgf.parray, rgIndices, &fld[i]);
        ::SafeArrayPutElement(rgv.parray, rgIndices, &val[i]);
        }
            m_pRs->Update(&rgf,&rgv);

    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

▶ **Build and test your application**

- Run your application and test the add, delete, and update buttons.

The solution code for this exercise is located in the folder \Labs\Ch08\Lab8.1\Ex01\Solution.

Notes

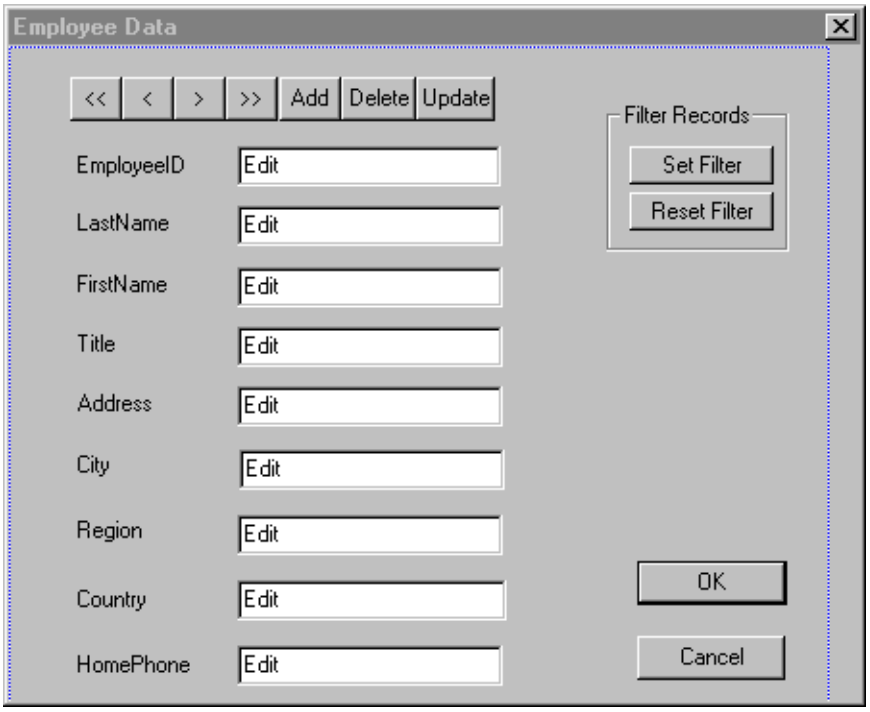
266 Exercise 2: Filtering Records

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch08\Lab8.1\Ex02.

In this exercise, you will use the **Filter** property of the recordset to filter records that satisfy criteria entered by the user.

▶ Add controls to CG_IDD_RECORDSET dialog box

- In the **CG_IDD_RECORDSET** dialog box, add buttons to implement filtering of records as shown in the following illustration.



Give the filter and reset filter controls the IDs **ID_FILTER** and **ID_RESETFILTER**, respectively.

▶ Create message map entries for the SetFilter and ResetFilter buttons

☐ 1. Open the RsCgDlg.h file and add the following entries after the //{{AFX_MSG message:

```
afx_msg void OnFilter();
afx_msg void OnResetFilter();
```

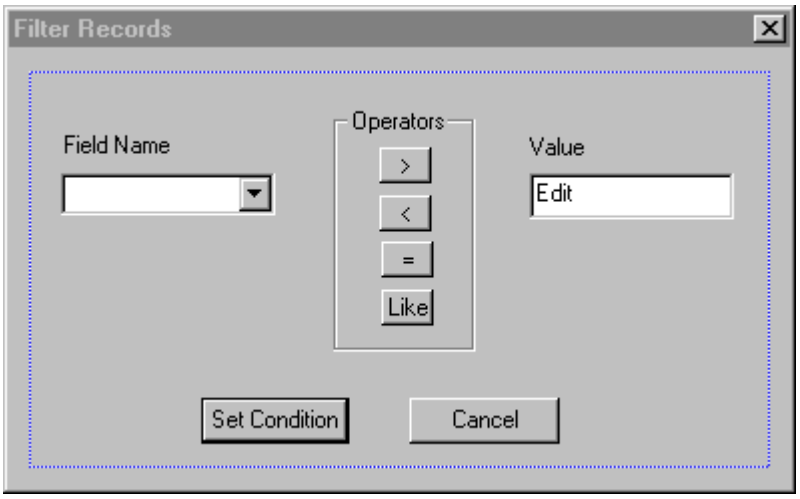
☐ 2. Insert message map entries for the ON_BN_CLICKED message in the RsCgDlg.cpp implementation file as follows:

```
ON_BN_CLICKED(ID_FILTER,OnFilter)
ON_BN_CLICKED(ID_RESETFILTER,OnResetFilter)
```

Notes

▶ Create a dialog box resource

In this procedure, you will create a dialog box that enables the user to specify criteria. The following illustration shows the dialog box you will create.



☐ 1. Use ResourceView to insert a new dialog box resource.

☐ 2. Set the **ID** and **Caption** properties of the dialog box as follows.

Property	Value
Caption	Filter Records
ID	IDD_QUERY

☐ 3. Add the following controls to the dialog box to create a dialog box as shown in the preceding illustration.

Control type	ID	Caption
Default command button	IDOK	Set Condition
Command button	IDCANCEL	Cancel
Static text	IDC_STATIC	Field Name
Combo box	ID_FIELD	None
Group box	IDC_STATIC	Operators
Command button	ID_GREATER	>
Command button	ID_LESSER	<
Command button	ID_EQUAL	=
Command button	ID_LIK	Like
Static text	IDC_STATIC	Value
Edit box	ID_VALUE	None

Notes

☐ 4. Open the **Combo Box Properties** dialog box, and click the **Data** tab.

☐ 5. In the **Enter listbox items** list box, enter the field names in the **Employees** database table.

▶ **Create a dialog class by using ClassWizard**

☐ 1. Invoke ClassWizard**.**

☐ 2. Select the **Create a new class** option, to create a new class for the **IDD_QUERY** dialog box and click **OK**.

☐ 3. In the **Name** box, type **CQuery** for the name of the associated C++ dialog class.

☐ 4. In the **Base** class box, select **CDialog**.

☐ 5. Click **OK**.

▶ **Add member variables to the dialog class**

In this procedure, you will create member variables for the new **CQuery** class.

☐ 1. Invoke ClassWizard.

☐ 2. On the **Member Variables** tab, set the class name to **CQuery** and add the following variables.

Control IDs	Type	Member
ID_FIELD	**CString**	m_Field
ID_VALUE	**CString**	m_Value

☐ 3. Open the Query.h file.

☐ 4. Create member variables to store the operator and criteria entered by the user as follows:

```
public:
char m_operator[7];
char cmd_str[51];
```

▶ **Create message map entries for command buttons on the IDD_QUERY dialog box**

☐ 1. Open Query.h and add the following entries after the //{{AFX_MSG message:

```
afx_msg void OnGreater();
afx_msg void OnLesser();
afx_msg void OnEqual();
afx_msg void OnLike();
afx_msg void OnSetCondition();
```

☐ 2. Insert message map entries for the ON_BN_CLICKED message in the **CQuery** implementation file as follows:

```
ON_BN_CLICKED(ID_GREATER,OnGreater)
ON_BN_CLICKED(ID_LESSER,OnLesser)
ON_BN_CLICKED(ID_EQUAL,OnEqual)
ON_BN_CLICKED(ID_LIK,OnLike)
ON_BN_CLICKED(IDOK,OnSetCondition)
```

Notes

▶ **Implement the OnGreater button handler**

- In the Query.cpp file, write the code for the **OnGreater** button handler as follows:

```
void CQuery::OnGreater()
{
    strcpy(m_operator, " > ");
}
```

Repeat the procedure to implement the **OnLesser**, **OnEqual**, and **OnLike** button handlers.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CQuery::OnGreater()
{
    strcpy(m_operator, " > ");
}

void CQuery::OnLesser()
{
    strcpy(m_operator, " < ");
}

void CQuery::OnEqual()
{
    strcpy(m_operator, " = ");
}

void CQuery::OnLike()
{
    strcpy(m_operator, " like ");
}
```

▶ **Implement the OnSetCondition button handler**

In this procedure, you will construct a criteria based on the field, operator, and the value entered by the user.

☐ 1. In the Query.cpp file, add the **OnSetCondition** button handler as follows:

```
void CQuery::OnSetCondition()
{
```

☐ 2. Invoke **UpdateData** with the argument **TRUE**.

Notes

☐ 3. Construct the criteria as follows:

```
if(m_Field != _T("") && m_Value != _T(""))
{
    strcpy(cmd_str,m_Field);
    strcat(cmd_str,m_operator);
    strcat(cmd_str,"'");
    strcat(cmd_str,m_Value);
    if(strcmp(m_operator," like ")==0)
        strcat(cmd_str, "*'");
    else
        strcat(cmd_str, "'");
}
CDialog::OnOK();
```

▶ **Implement the OnFilter button handler**

In this procedure, you will invoke a dialog box to accept criteria from the user and then filter records that satisfy the specified criteria.

☐ 1. In the RsCgDlg.cpp file, add the **OnFilter** handler as follows:

```
void CRsCgDlg::OnFilter()
{
```

☐ 2. Clear the values of the current record displayed on the form as follows:

```
m_strDlgAddress = _T("");
m_strDlgCity = _T("");
m_strDlgCountry = _T("");
m_lDlgEmployeeID = 0;
m_strDlgFirstName = _T("");
m_strDlgHomePhone = _T("");
m_strDlgLastName = _T("");
m_strDlgRegion = _T("");
m_strDlgTitle = _T("");

UpdateData(FALSE);
```

☐ 3. In ClassView, right click **CRsCgDlg**, and create a protected member variable m_qDlg of type **CQuery**.

☐ 4. Invoke the dialog box to accept the criteria as follows:

```
int nRes = m_qDlg.DoModal();
```

Notes

☐ 5. Filter records that satisfy the specified criteria as follows:

```
try
{
    m_pRs->Filter = m_qDlg.cmd_str;
    if(m_pRs->BOF || m_pRs->EndOfFile)
    {
        MessageBox("No Matching Records");
        m_pRs->Filter="";
    }
    else
        OnBtnMoveFirst();
}
```

☐ 6. Write the code for the **catch** handler.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CRsCgDlg::OnFilter()
{
    m_strDlgAddress = _T("");
    m_strDlgCity = _T("");
    m_strDlgCountry = _T("");
    m_lDlgEmployeeID = 0;
    m_strDlgFirstName = _T("");
    m_strDlgHomePhone = _T("");
    m_strDlgLastName = _T("");
    m_strDlgRegion = _T("");
    m_strDlgTitle = _T("");
    UpdateData(FALSE);
    int nRes = m_qDlg.DoModal();

    try
    {
        m_pRs->Filter = m_qDlg.cmd_str;
        if(m_pRs->BOF | m_pRs->EndOfFile)
        {
            MessageBox("No Matching Records");
            m_pRs->Filter="";

        }
        else
        OnBtnMoveFirst();


    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

Notes

▶ Implement the OnResetFilter button handler

In this procedure, you will clear the **Filter** property so that all the records in the recordset will appear.

- Write the code for the **OnResetFilter** handler as follows:

```
void CRsCgDlg::OnResetFilter()
{
    m_pRs->Filter="";

}
```

▶ Build and test your application

- Run your application and test the **Filter** and **ResetFilter** buttons.

The solution code for this exercise is located in the folder \Labs\Ch08\Lab8.1\Ex02\Solution.

Notes